

BIKE Tile Min-Sum 译码器调度与架构说明

1 固定窗口 tile 调度图

图 1 展示 `tile_scheduler` 生成的固定 tile window 调度。C2V 路径填充当前 tile buffer, V2C 路径排空前一个 tile 对应的 active buffer。

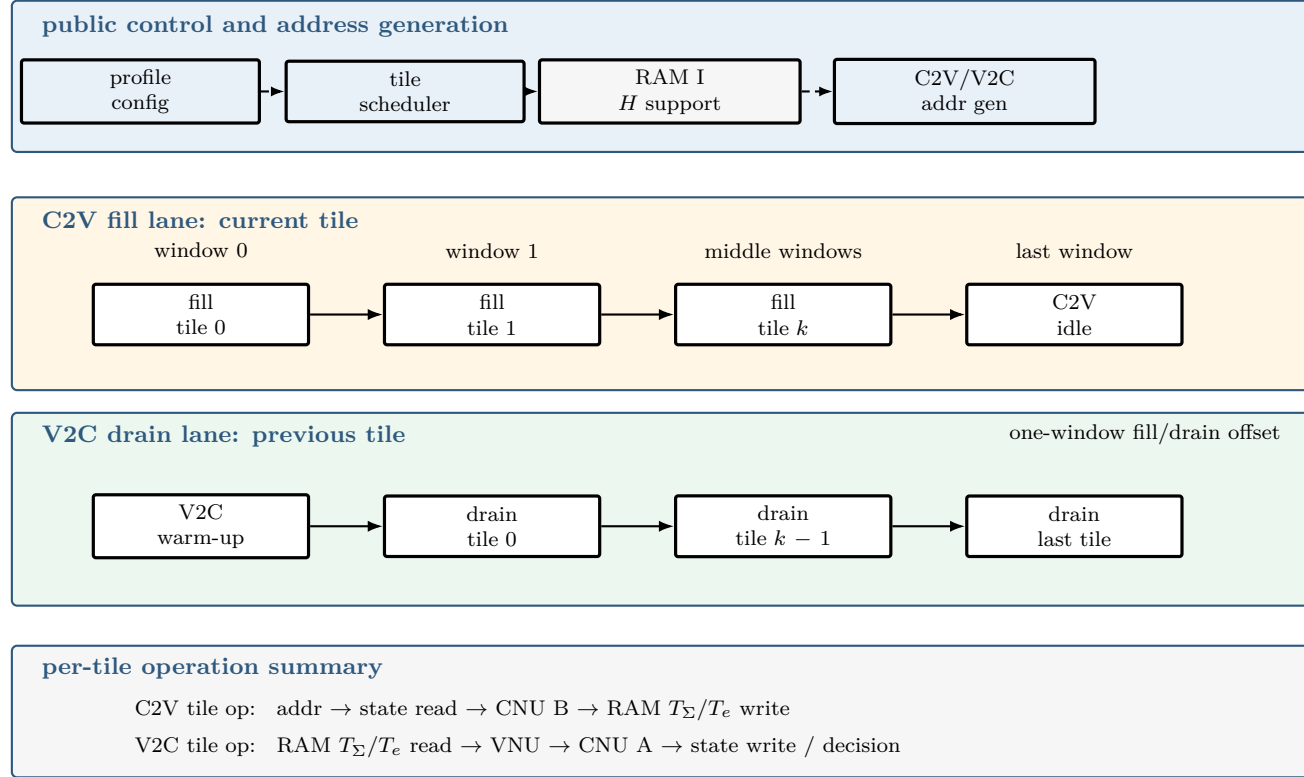


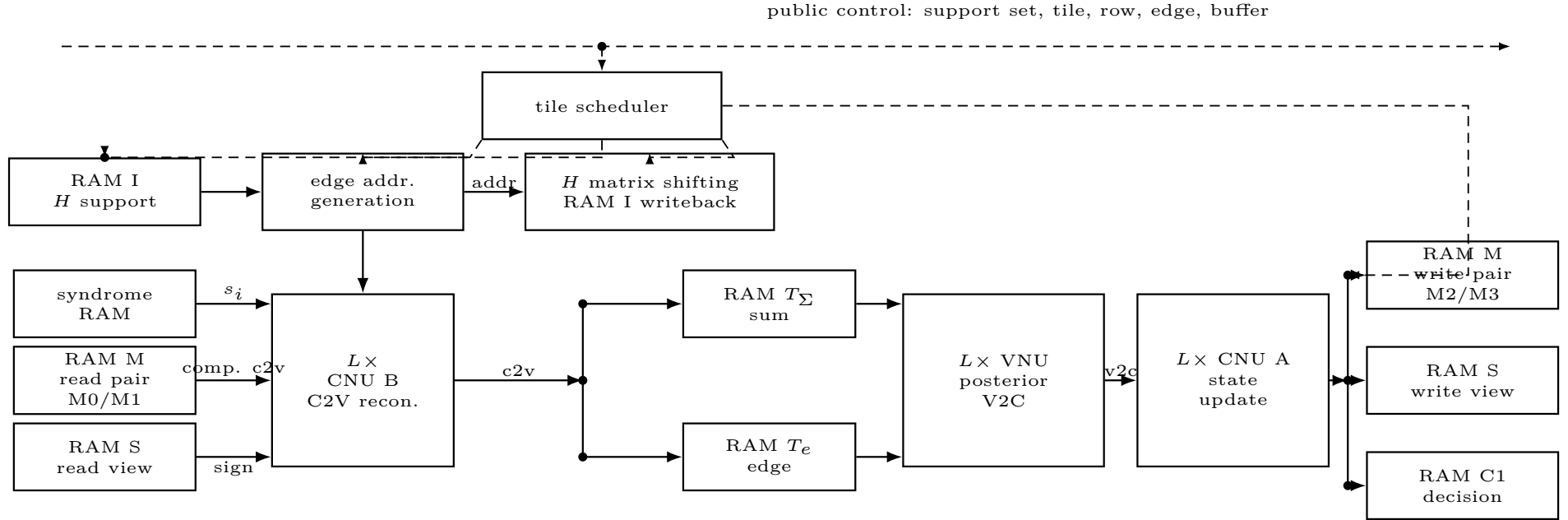
Figure 1: Fixed-window tile schedule of the BIKE min-sum decoder.

2 顶层硬件架构图

图 2 给出模块级译码器数据通路。该图保留 rtl/decoder_top.sv 中最关键的存储和计算模块；控制逻辑只用带标注的输入线表示。

L lanes process one tile window; RAM M stores $(\min_1, \min_2, \min_id, \text{sign_xor})$. RAM T_{Σ}/T_e hold the active tile C2V sums and edge messages.

Figure 2: Top-level hardware architecture of the tile-based BIKE min-sum decoder.



RAM M stores $(\min_1, \min_2, \min_id, \text{sign_xor})$; RAM T_{Σ}/T_e store tile C2V sums and edge messages.

Figure 3: Orthogonal datapath view of the tile-based BIKE min-sum decoder.

3 架构说明

3.1 调度图简写说明

图中每一列表示一个固定 tile window。window 0 用来填充第一个 tile buffer；中间窗口将 tile k 的 C2V 填充与 tile $k-1$ 的 V2C 排空重叠执行；最后一个窗口排空末尾 tile。下方 operation summary 给出每个 tile 内部的数据通路阶段：state read 和 state write 是同一组 RAM M/RAM S 在 C2V 与 V2C 阶段的访问视图；RAM T_Σ /RAM T_e 提供 tile buffer 的 fill/active 双缓冲视图。RAM M 表示压缩 check state 存储，内容为 $(\min_1, \min_2, \min_id, \text{sign_xor})$ 。RAM S 存储 CNU B 重建 C2V 时需要的 V2C 符号。RAM T_Σ 存储 tile 内 raw C2V 总和，RAM T_e 存储排除自身时要减去的单边 C2V。CNU B 负责重建 C2V，VNU 负责 posterior 和 V2C 变量节点更新，CNU A 负责写入下一轮压缩 check state。

3.2 顶层组织

译码器输入为 syndrome 和 QC-MDPC 校验矩阵每个 circulant block 的第一列支撑集。ram_i 保存 $N_0 \times W$ 个 base row，ram_syndrome 保存 r 位 syndrome。统一参数配置下，decoder_profile_config 根据公开 profile 输出 R 、 W 、tile 数、row bank 深度、 C 和 α 的两个移位量。主译码由 tile_scheduler 驱动，调度顺序只依赖公开参数。

局部变量列按 tile 组织：

$$C_{\text{tile}} = \min(R, \text{BIKE_C_TILE}), \quad Q_{\text{base}} = \left\lceil \frac{C_{\text{tile}}}{L} \right\rceil, \quad Q_{\text{tile}} = Q_{\text{base}} + 3.$$

调度器对每个 tile、每个第一列支撑项 one_idx、每个 q_seq 执行固定窗口。edge_addr_gen 把 base row、tile index 和 lane index 展开为实际 row、变量列、edge id、row bank 和 tile offset。当循环行号跨越 $R-1 \rightarrow 0$ 时，guard 周期承载固定的 micro-cycle 拆分。

3.3 C2V 流水

C2V 路径从压缩 check state 重建校验节点到变量节点的消息。每个有效 lane 读取：

$$\text{RAM M} : (\min_1, \min_2, \min_id, \text{sign_xor}), \quad \text{RAM S} : \text{sign}(u_{i,j}), \quad \text{syndrome} : s_i.$$

cnu_b 根据 edge_id 选择 min1 或 min2，并计算

$$\text{sign}(v_{i,j}) = \text{sign_xor} \oplus \text{sign}(u_{i,j}) \oplus s_i.$$

输出的 sign-magnitude C2V 被转换为 two's complement 原始值。该值写入两个 tile buffer：ram_t_accum 累加同一变量列收到的 raw C2V 总和，ram_t 保存单条边的 raw C2V 消息。one_idx==0 的访问把该变量列的累加基值视为 0。

3.4 V2C 流水

V2C 路径读取前一个 tile 的 ram_t_accum 和 ram_t。vnu_update 在每 lane 计算：

$$\tilde{\gamma}_j = C + \alpha \sum_{i \in \mathcal{N}(j)} v_{i,j}, \quad u_{i,j} = C + \alpha \left(\sum_{i' \in \mathcal{N}(j)} v_{i',j} - v_{i,j} \right).$$

其中 $\alpha = 2^{-s_0} + 2^{-s_1}$ ，RTL 通过两个左移到定点小数域后的加法实现，并在回到整数域时舍入。V2C 结果饱和为 sign-magnitude 消息后送入 cnu_a。cnu_a 增量更新当前 row 的压缩状态：符号进入 sign_xor，幅度参与 min1/min2/min_id 比较。更新后的状态写回 ram_m 的 write pair，V2C 符号写入 ram_s，供后续 C2V 重建使用。最后一轮中，one_idx==0 时 posterior 符号写入 ram_c1 作为错误估计位。

3.5 存储与双缓冲

核心状态分为五类。ram_m 是 banked compressed check-state storage，包含两个 pair；C2V 从 read pair 读取上一轮形成的状态，V2C 向 write pair 写入本轮形成的状态。ram_s 存储每条 V2C 消息的符号位，以 tile word 粒度读写。ram_t_accum 是 double-buffered raw C2V sum storage，服务于 C2V 的读-累加-写回和 V2C 的求和读取。ram_t 是 double-buffered edge cache，保存排除自身时需要减去的单条 C2V。ram_c1 保存最终错误估计并提供外部读口。

tile buffer 使用 fill_buf 和 active_buf 交替选择。第 0 个窗口只填充 tile 0；中间窗口让 C2V 处理当前 tile、V2C 处理前一个 tile；最后一个窗口排空末尾 tile。该组织让每轮迭代包含固定的初始化和窗口数量。

3.6 固定时间预算

每轮先初始化 write pair 的压缩 check state, 周期数为

$$T_{\text{clear}} = \left\lceil \frac{R}{L} \right\rceil.$$

每个 tile 窗口周期数为

$$T_{\text{tile}} = W \cdot Q_{\text{tile}}.$$

总 tile 数为 $T_{\text{total}} = N_0 \lceil R/C_{\text{tile}} \rceil$, 每轮译码周期为

$$T_{\text{iter}} = T_{\text{clear}} + (T_{\text{total}} + 1) \cdot T_{\text{tile}},$$

完整译码周期为

$$T_{\text{decode}} = I_{\text{max}} \cdot T_{\text{iter}}.$$

这些量由公开 profile、 L 和 C_{tile} 决定。H 的 base row、syndrome、错误模式和收敛行为只影响数据值、lane mask 或 row 地址, 不改变主译码窗口数量。

References

- [1] J. Cai and X. Zhang, “Low-complexity parallel min-sum medium-density parity-check decoder for McEliece cryptosystem,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 12, pp. 5327–5339, Dec. 2023.